Capítulo 09 - Agente Escriba (Scribe Agent)

9.1. Introdução ao Agente Escriba

O **Agente Escriba** é o componente criativo do Projeto Quimera. Ele recebe a estratégia de persuasão do Motor de Persuasão e **transforma em uma mensagem real e natural** usando Large Language Models (LLMs).

9.1.1. Responsabilidades do Agente Escriba

1. **Receber estratégia de persuasão:**
   - Princípio, tom, gatilhos, afinidades
   - Estrutura da mensagem (gancho, conexão, valor, CTA)
2. **Gerar mensagem usando LLM:**
   - Construir prompt detalhado
   - Chamar API do LLM (GPT-4, Claude, etc.)
   - Obter mensagem gerada
3. **Validar mensagem:**
   - Verificar comprimento
   - Verificar tom
   - Verificar personalização
   - Detectar problemas (clichês, linguagem inapropriada)
4. **Refinar mensagem (se necessário):**
   - Regenerar com ajustes
   - Limitar tentativas (máximo 3)
5. **Salvar mensagem aprovada:**
   - Armazenar no banco de dados
   - Emitir evento MensagemGerada

9.1.2. Desafios Principais

1. Naturalidade

LLMs podem gerar texto que soa robótico ou genérico. Precisamos de prompts bem construídos para gerar mensagens naturais.

2. Consistência

LLMs são estocásticos (aleatórios). A mesma entrada pode gerar saídas diferentes. Precisamos controlar temperatura e validar saída.

3. Custo

Chamadas de API de LLMs custam dinheiro. GPT-4 custa ~$0.03 por 1K tokens. Precisamos otimizar uso.

4. Latência

LLMs podem levar 2-10 segundos para responder. Precisamos de estratégias de timeout e retry.

9.2. Large Language Models (LLMs)

9.2.1. O que são LLMs?

Large Language Models são modelos de IA treinados em bilhões de palavras para entender e gerar linguagem humana.

Exemplos:

- **GPT-4** (OpenAI) - Mais capaz, mais caro
- **GPT-4o-mini** (OpenAI) - Mais rápido, mais barato
- **Claude 3 Opus** (Anthropic) - Muito capaz, seguro
- **Claude 3 Sonnet** (Anthropic) - Balanceado
- **Llama 3** (Meta) - Open source, pode hospedar localmente

9.2.2. Comparação de Modelos

Modelo	Custo (1M tokens)	Velocidade	Qualidade	Contexto
GPT-4	$30 input / $60 output	Lento	Excelente	128K
GPT-4o-mini	$0.15 input / $0.60 output	Rápido	Boa	128K
Claude 3 Opus	$15 input / $75 output	Médio	Excelente	200K
Claude 3 Sonnet	$3 input / $15 output	Rápido	Muito Boa	200K
Llama 3 70B	Grátis (self-hosted)	Médio	Boa	8K

Recomendação: Usar **GPT-4o-mini** para produção (bom custo-benefício) e **GPT-4** para casos especiais.

9.2.3. Configuração da API

Python

```python
from openai import OpenAI
import os

# Inicializar cliente
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def gerar_texto(prompt, modelo="gpt-4o-mini", temperatura=0.7,
max_tokens=500):
    """
    Gera texto usando OpenAI API.

    Args:
        prompt: Prompt de entrada
        modelo: Modelo a usar
        temperatura: Criatividade (0-2, recomendado 0.7-1.0)
        max_tokens: Máximo de tokens na resposta

    Returns:
        str: Texto gerado
    """
    try:
        response = client.chat.completions.create(
            model=modelo,
            messages=[
                {"role": "system", "content": "Você é um assistente
especializado em comunicação interpessoal."},
                {"role": "user", "content": prompt}
            ],
            temperature=temperatura,
            max_tokens=max_tokens,
            top_p=1.0,
            frequency_penalty=0.5,  # Reduz repetição
            presence_penalty=0.3    # Incentiva novos tópicos
        )

        texto_gerado = response.choices[0].message.content
        tokens_usados = response.usage.total_tokens
        custo = calcular_custo(tokens_usados, modelo)

        return {
```

```
            'texto': texto_gerado,
            'tokens': tokens_usados,
            'custo_usd': custo
        }

    except Exception as e:
        print(f"Erro ao gerar texto: {e}")
        return None


def calcular_custo(tokens, modelo):
    """Calcula custo em USD baseado no modelo."""
    custos = {
        'gpt-4': (0.03 / 1000, 0.06 / 1000),  # (input, output)
        'gpt-4o-mini': (0.00015 / 1000, 0.0006 / 1000),
        'claude-3-opus': (0.015 / 1000, 0.075 / 1000),
        'claude-3-sonnet': (0.003 / 1000, 0.015 / 1000)
    }

    # Simplificação: assumir 50% input, 50% output
    custo_input, custo_output = custos.get(modelo, (0, 0))
    return (tokens * 0.5 * custo_input) + (tokens * 0.5 * custo_output)
```

9.3. Prompt Engineering

9.3.1. O que é Prompt Engineering?

Prompt Engineering é a arte de construir instruções (prompts) que fazem o LLM gerar exatamente o que você quer.

Componentes de um bom prompt:

1. **Contexto:** Quem é o LLM? Qual é o objetivo?
2. **Tarefa:** O que exatamente fazer?
3. **Restrições:** O que evitar? Limites?
4. **Exemplos:** Mostrar exemplos do resultado desejado
5. **Formato:** Como estruturar a saída?

9.3.2. Estrutura do Prompt para Mensagens

Plain Text

```
[CONTEXTO]
Você é um especialista em comunicação interpessoal e sedução ética.
Sua tarefa é escrever uma mensagem de abordagem para alguém no Instagram.

[PERFIL DO DESTINATÁRIO]
- Nome: {nome}
- Idade: {idade}
- Interesses: {interesses}
- Personalidade: {tracos_personalidade}
- Bio: "{biografia}"

[ESTRATÉGIA DE PERSUASÃO]
- Princípio: {principio}
- Tom: {tom}
- Gatilhos emocionais: {gatilhos}
- Ponto de afinidade principal: {afinidade}

[ESTRUTURA DA MENSAGEM]
1. Gancho: {gancho_sugerido}
2. Conexão: {conexao_sugerida}
3. Valor: {valor_sugerido}
4. Call-to-Action: {cta_sugerido}

[RESTRIÇÕES]
- Máximo 500 caracteres
- Tom: {tom}
- Usar emojis: {usar_emojis}
- NÃO usar clichês como "sua foto é linda"
- NÃO ser muito direto ou sexual
- NÃO parecer desesperado
- SER específico e personalizado
- SER genuíno e natural

[EXEMPLOS DE MENSAGENS BOAS]
{exemplos}

[TAREFA]
Escreva UMA mensagem de abordagem seguindo a estratégia e estrutura acima.
Retorne APENAS a mensagem, sem explicações adicionais.
```

9.3.3. Implementação do Construtor de Prompt

Python

```python
def construir_prompt(perfil, estrategia, usuario):
    """
```

```
    Constrói prompt detalhado para geração de mensagem.

    Args:
        perfil: Dados do perfil analisado
        estrategia: Estratégia de persuasão
        usuario: Dados do usuário

    Returns:
        str: Prompt completo
    """
    # Extrair dados
    nome = perfil.get('nome_completo', 'pessoa')
    idade = perfil.get('idade_aparente', 'desconhecida')
    interesses = ', '.join(perfil.get('interesses', []))
    personalidade = formatar_personalidade(perfil.get('personalidade', {}))
    biografia = perfil.get('biografia', '')

    principio = estrategia.get('principio')
    tom = estrategia.get('tom')
    gatilhos = ', '.join(estrategia.get('gatilhos', []))
    afinidade = estrategia.get('afinidade_principal', {})

    estrutura = estrategia.get('estrutura', {})
    gancho = estrutura.get('gancho', '')
    conexao = estrutura.get('conexao', '')
    valor = estrutura.get('valor', '')
    cta = estrutura.get('cta', '')

    usar_emojis = 'Sim' if tom in ['casual', 'amigavel'] else 'Não'

    # Obter exemplos
    exemplos = obter_exemplos_mensagens(principio, tom)

    # Construir prompt
    prompt = f"""[CONTEXTO]
Você é um especialista em comunicação interpessoal e sedução ética.
Sua tarefa é escrever uma mensagem de abordagem para alguém no Instagram.

[PERFIL DO DESTINATÁRIO]
- Nome: {nome}
- Idade: {idade}
- Interesses: {interesses}
- Personalidade: {personalidade}
- Bio: "{biografia}"

[ESTRATÉGIA DE PERSUASÃO]
- Princípio: {principio}
- Tom: {tom}
```

```
- Gatilhos emocionais: {gatilhos}
- Ponto de afinidade principal: {afinidade.get('descricao', '')}

[ESTRUTURA DA MENSAGEM]
1. Gancho: {gancho}
2. Conexão: {conexao}
3. Valor: {valor}
4. Call-to-Action: {cta}

[RESTRIÇÕES]
- Máximo 500 caracteres
- Tom: {tom}
- Usar emojis: {usar_emojis}
- NÃO usar clichês como "sua foto é linda"
- NÃO ser muito direto ou sexual
- NÃO parecer desesperado
- SER específico e personalizado
- SER genuíno e natural

[EXEMPLOS DE MENSAGENS BOAS]
{exemplos}

[TAREFA]
Escreva UMA mensagem de abordagem seguindo a estratégia e estrutura acima.
Retorne APENAS a mensagem, sem explicações adicionais.
"""

    return prompt


def formatar_personalidade(personalidade):
    """Formata traços de personalidade para o prompt."""
    tracos = []

    if personalidade.get('extroversao', 0) > 0.6:
        tracos.append('extrovertida')
    if personalidade.get('abertura', 0) > 0.6:
        tracos.append('aberta a experiências')
    if personalidade.get('amabilidade', 0) > 0.6:
        tracos.append('amigável')

    return ', '.join(tracos) if tracos else 'equilibrada'


def obter_exemplos_mensagens(principio, tom):
    """Retorna exemplos de mensagens para o princípio e tom."""
    exemplos_db = {
        ('afeicao', 'casual'): """
```

```
Exemplo 1: "E aí! Adorei suas fotos de viagem! Também sou viciado em
conhecer lugares novos. Já foi em Ouro Preto? Tenho umas dicas massa se
quiser! 🗺️"

Exemplo 2: "Opa! Vi que você curte fotografia também. Aquela foto do pôr do
sol ficou perfeita! Você usa qual câmera?"
""",
        ('afeicao', 'amigavel'): """
Exemplo 1: "Oi! Adorei seu perfil! Percebi que a gente tem muito em comum,
especialmente o amor por viagens. Sua foto em Machu Picchu ficou incrível!
Você tem mais dicas de lugares na América do Sul?"

Exemplo 2: "Olá! Vi que você pratica yoga. Também pratico há alguns anos e
adoro! Você frequenta algum estúdio em Varginha?"
""",
        # ... mais exemplos para outros princípios e tons
    }

    return exemplos_db.get((principio, tom), "")
```

9.3.4. Técnicas Avançadas de Prompt Engineering

1. Few-Shot Learning

Fornecer exemplos no prompt para o LLM aprender o padrão.

Python

```
prompt = """
Exemplos de mensagens boas:

Entrada: Perfil que curte fotografia e viagens
Saída: "Adorei suas fotos de viagem! Aquela de Machu Picchu ficou perfeita.
Você usa qual câmera?"

Entrada: Perfil que pratica yoga
Saída: "Vi que você pratica yoga! Também pratico há anos. Você frequenta
algum estúdio aqui em Varginha?"

Agora, gere uma mensagem para:
Entrada: Perfil que curte música e festivais
Saída:
"""
```

2. Chain-of-Thought Prompting

Pedir ao LLM para "pensar em voz alta" antes de gerar a resposta final.

```python
prompt = """
Antes de escrever a mensagem, pense:
1. Qual é o interesse principal desta pessoa?
2. Qual gatilho emocional usar?
3. Como criar conexão genuína?

Agora, escreva a mensagem:
"""
```

3. Role Prompting

Atribuir um papel específico ao LLM.

```python
system_message = """
Você é um coach de relacionamentos com 10 anos de experiência.
Você é conhecido por criar mensagens de abordagem autênticas e eficazes.
Seu estilo é caloroso, genuíno e respeitoso.
"""
```

9.4. Geração de Mensagens

9.4.1. Fluxo Completo de Geração

```python
def gerar_mensagem(perfil, estrategia, usuario):
    """
    Gera mensagem completa usando LLM.

    Returns:
        dict: Mensagem gerada com metadados
    """
    # 1. Construir prompt
    prompt = construir_prompt(perfil, estrategia, usuario)

    # 2. Gerar mensagem
    resultado = gerar_texto(
        prompt=prompt,
        modelo="gpt-4o-mini",
        temperatura=0.8,  # Criatividade moderada
        max_tokens=300
```

```
    )

    if not resultado:
        return None

    mensagem = resultado['texto'].strip()

    # 3. Validar mensagem
    valida, problemas = validar_mensagem(mensagem, estrategia)

    if not valida:
        print(f"Mensagem inválida: {problemas}")
        # Tentar regenerar (máximo 3 tentativas)
        return regenerar_mensagem(perfil, estrategia, usuario, problemas,
tentativa=1)

    # 4. Retornar mensagem aprovada
    return {
        'conteudo': mensagem,
        'modelo_usado': 'gpt-4o-mini',
        'tokens_usados': resultado['tokens'],
        'custo_usd': resultado['custo_usd'],
        'validada': True,
        'tentativas': 1
    }


def regenerar_mensagem(perfil, estrategia, usuario, problemas, tentativa,
max_tentativas=3):
    """Regenera mensagem com ajustes baseados nos problemas."""
    if tentativa > max_tentativas:
        print("Máximo de tentativas atingido")
        return None

    # Ajustar prompt com feedback
    prompt = construir_prompt(perfil, estrategia, usuario)
    prompt += f"\n\n[FEEDBACK DA TENTATIVA ANTERIOR]\nProblemas encontrados:
{', '.join(problemas)}\nCorrija esses problemas na nova mensagem.\n"

    # Gerar novamente
    resultado = gerar_texto(prompt, temperatura=0.9)  # Aumentar criatividade

    if not resultado:
        return None

    mensagem = resultado['texto'].strip()

    # Validar novamente
```

```
    valida, novos_problemas = validar_mensagem(mensagem, estrategia)

    if not valida:
        return regenerar_mensagem(perfil, estrategia, usuario,
novos_problemas, tentativa + 1)

    return {
        'conteudo': mensagem,
        'modelo_usado': 'gpt-4o-mini',
        'tokens_usados': resultado['tokens'],
        'custo_usd': resultado['custo_usd'],
        'validada': True,
        'tentativas': tentativa + 1
    }
```

9.5. Validação de Mensagens

9.5.1. Critérios de Validação

Python

```python
def validar_mensagem(mensagem, estrategia):
    """
    Valida mensagem gerada.

    Returns:
        tuple: (bool, list) - (é_valida, lista_de_problemas)
    """
    problemas = []

    # 1. Verificar comprimento
    if len(mensagem) > 500:
        problemas.append("Mensagem muito longa (>500 caracteres)")

    if len(mensagem) < 50:
        problemas.append("Mensagem muito curta (<50 caracteres)")

    # 2. Verificar clichês
    cliches = [
        'sua foto é linda',
        'você é muito bonita',
        'gostosa',
        'princesa',
        'deusa',
        'perfeita'
```

```
    ]

    mensagem_lower = mensagem.lower()
    for cliche in cliches:
        if cliche in mensagem_lower:
            problemas.append(f"Contém clichê: '{cliche}'")

    # 3. Verificar tom
    tom_esperado = estrategia.get('tom')
    if not verificar_tom(mensagem, tom_esperado):
        problemas.append(f"Tom não corresponde ao esperado: {tom_esperado}")

    # 4. Verificar personalização
    if not verificar_personalizacao(mensagem, estrategia):
        problemas.append("Mensagem muito genérica, falta personalização")

    # 5. Verificar linguagem inapropriada
    palavras_inapropriadas = ['sexo', 'tesão', 'gostosa', 'delícia']
    for palavra in palavras_inapropriadas:
        if palavra in mensagem_lower:
            problemas.append(f"Linguagem inapropriada: '{palavra}'")

    # 6. Verificar se tem pergunta/CTA
    if not ('?' in mensagem or any(palavra in mensagem_lower for palavra in
['vamos', 'que tal', 'bora'])):
        problemas.append("Falta call-to-action ou pergunta")

    return (len(problemas) == 0, problemas)


def verificar_tom(mensagem, tom_esperado):
    """Verifica se o tom da mensagem corresponde ao esperado."""
    indicadores_tom = {
        'casual': ['e aí', 'opa', 'massa', 'bora', 'tipo'],
        'amigavel': ['oi', 'olá', 'adorei', 'legal', 'seria legal'],
        'confiante': ['vi', 'percebi', 'vamos', 'tenho certeza'],
        'prestativo': ['posso', 'ajudar', 'compartilhar', 'dicas']
    }

    mensagem_lower = mensagem.lower()
    indicadores = indicadores_tom.get(tom_esperado, [])

    # Verificar se pelo menos um indicador está presente
    return any(indicador in mensagem_lower for indicador in indicadores)


def verificar_personalizacao(mensagem, estrategia):
    """Verifica se a mensagem é personalizada."""
```

```
    afinidade = estrategia.get('afinidade_principal', {})

    if not afinidade:
        return False

    # Verificar se menciona o interesse/afinidade
    tipo_afinidade = afinidade.get('tipo', '').lower()
    descricao = afinidade.get('descricao', '').lower()

    mensagem_lower = mensagem.lower()

    return tipo_afinidade in mensagem_lower or any(palavra in mensagem_lower
for palavra in descricao.split())
```

9.6. Otimizações

9.6.1. Cache de Mensagens Similares

Para perfis muito similares, podemos reutilizar mensagens (com pequenas adaptações).

Python

```
import hashlib

def gerar_hash_perfil(perfil):
    """Gera hash baseado em características do perfil."""
    caracteristicas = f"{perfil.get('interesses',
[])}_{perfil.get('personalidade', {})}"
    return hashlib.md5(caracteristicas.encode()).hexdigest()


def buscar_mensagem_cache(perfil):
    """Busca mensagem similar no cache."""
    hash_perfil = gerar_hash_perfil(perfil)

    # Buscar no Redis
    mensagem_cache = redis_client.get(f"mensagem:{hash_perfil}")

    if mensagem_cache:
        return mensagem_cache.decode('utf-8')

    return None


def salvar_mensagem_cache(perfil, mensagem):
    """Salva mensagem no cache."""
```

```
    hash_perfil = gerar_hash_perfil(perfil)
    redis_client.setex(f"mensagem:{hash_perfil}", 3600, mensagem)  # TTL 1
hora
```

9.6.2. Batch Generation

Gerar múltiplas mensagens em paralelo para reduzir latência total.

Python

```python
import asyncio
from openai import AsyncOpenAI

async_client = AsyncOpenAI(api_key=os.getenv("OPENAI_API_KEY"))

async def gerar_mensagem_async(prompt):
    """Gera mensagem de forma assíncrona."""
    response = await async_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "Você é um especialista em
comunicação."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.8,
        max_tokens=300
    )

    return response.choices[0].message.content


async def gerar_mensagens_batch(perfis, estrategias, usuario):
    """Gera mensagens para múltiplos perfis em paralelo."""
    prompts = [construir_prompt(p, e, usuario) for p, e in zip(perfis,
estrategias)]

    tasks = [gerar_mensagem_async(prompt) for prompt in prompts]
    mensagens = await asyncio.gather(*tasks)

    return mensagens


# Uso
perfis = [perfil1, perfil2, perfil3]
estrategias = [estrategia1, estrategia2, estrategia3]
```

```
mensagens = asyncio.run(gerar_mensagens_batch(perfis, estrategias, usuario))
```

9.6.3. Fallback para Modelos Mais Baratos

Se GPT-4 falhar ou estiver muito caro, usar modelo mais barato como fallback.

Python

```python
def gerar_mensagem_com_fallback(perfil, estrategia, usuario):
    """Gera mensagem com fallback para modelos mais baratos."""
    modelos = ['gpt-4o-mini', 'gpt-3.5-turbo']

    for modelo in modelos:
        try:
            prompt = construir_prompt(perfil, estrategia, usuario)
            resultado = gerar_texto(prompt, modelo=modelo)

            if resultado:
                mensagem = resultado['texto'].strip()
                valida, _ = validar_mensagem(mensagem, estrategia)

                if valida:
                    return {
                        'conteudo': mensagem,
                        'modelo_usado': modelo,
                        'tokens_usados': resultado['tokens'],
                        'custo_usd': resultado['custo_usd']
                    }
        except Exception as e:
            print(f"Erro com modelo {modelo}: {e}")
            continue

    return None
```

9.7. Monitoramento e Métricas

9.7.1. Métricas Importantes

Python

```python
metricas_escriba = {
    'mensagens_geradas': 1000,
    'mensagens_validadas': 850,
```

```
    'taxa_validacao': 0.85,  # 85%
    'tentativas_media': 1.2,
    'tokens_totais_usados': 250000,
    'custo_total_usd': 3.75,
    'custo_medio_por_mensagem': 0.00375,
    'tempo_medio_geracao_segundos': 3.2,
    'modelo_mais_usado': 'gpt-4o-mini'
}
```

9.7.2. Logging de Gerações

Python

```python
import structlog

logger = structlog.get_logger()

def logar_geracao(perfil_id, mensagem, resultado):
    """Loga geração de mensagem."""
    logger.info(
        "mensagem_gerada",
        perfil_id=perfil_id,
        comprimento=len(mensagem),
        modelo=resultado['modelo_usado'],
        tokens=resultado['tokens_usados'],
        custo_usd=resultado['custo_usd'],
        tentativas=resultado['tentativas'],
        validada=resultado['validada']
    )
```

9.8. Considerações Éticas

9.8.1. Transparência

Questão: Devemos revelar que a mensagem foi gerada por IA?

Resposta: Não necessariamente. A mensagem é baseada em análise real do perfil e estratégia genuína. O LLM é apenas uma ferramenta de escrita, como um corretor ortográfico avançado.

9.8.2. Autenticidade

Regra: A mensagem deve refletir verdades sobre o usuário. Não inventar afinidades ou experiências.

9.8.3. Respeito

Regra: Nunca gerar mensagens:

- Sexuais ou inapropriadas
- Manipuladoras ou enganosas
- Que exploram vulnerabilidades
- Que violam privacidade

9.9. Conclusão do Capítulo

Este capítulo cobriu o **Agente Escriba** em profundidade:

- **LLMs:** Comparação de modelos, configuração de API
- **Prompt Engineering:** Estrutura, técnicas avançadas (few-shot, chain-of-thought, role prompting)
- **Geração:** Fluxo completo, regeneração com feedback
- **Validação:** 6 critérios de validação, verificação de tom e personalização
- **Otimizações:** Cache, batch generation, fallback
- **Monitoramento:** Métricas e logging
- **Ética:** Transparência, autenticidade, respeito

No próximo capítulo, veremos o **Agente de Execução**, que envia as mensagens geradas de forma automatizada.

Próximo Capítulo: Capítulo 10 - Agente de Execução

Referências:

- OpenAI API Documentation: https://platform.openai.com/docs/
- Anthropic Claude Documentation: https://docs.anthropic.com/
- Prompt Engineering Guide: https://www.promptingguide.ai/
- Best Practices for Prompt Engineering: https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-openai-api